

LLM-aided Human-Robot Interfacing for A Prompt-to-Control Application

H.P. Madushanka*, *Student Member, IEEE*, Sumudu Samarakoon*[†], *Member, IEEE*,
Rafaela Scaciota*[†], *Member, IEEE*, and Mehdi Bennis* *Fellow, IEEE*

*Centre for Wireless Communication, University of Oulu, Finland

[†]Infotech Oulu, University of Oulu, Finland

email: {madushanka.hewapathirana, sumudu.samarakoon, rafaela.scaciotatimoesdasilva, mehdi.bennis}@oulu.fi

Abstract—This paper presents a novel solution to interface and control a mobile robot with human operators leveraging a custom large language model (LLM) hosted on a server. Here, operators can provide commands as freeform text to the server via a user interface without a need of any prior knowledge on the machine language. In the server, a LLM that has been customized with the knowledge of the environment is used to break down the directives into a high-level plan in terms of i) Go where? ii) Find what? and iii) Do what?, which are then shared with the robot. Upon receiving the plan, the robot autonomously navigates to the designated location and identifies target objects utilizing an optimized tiny artificial intelligence (AI) model to ensure high performance in real-time. The demo validates the effectiveness of generating accurate plans even for the broader commands within the context and the ability to perform the task in real-time.

Index Term—LLM, Human-robot interfacing, Autonomous navigation and object detection

I. INTRODUCTION

As autonomous robots become increasingly common in human-centric environments, enabling seamless and intuitive human-robot communication remains a challenge. Traditional robot control systems often rely on structured data inputs or remote control methods, which are inconvenient for the non-experts [1]. In contrast, leveraging large language model (LLM) allows robots to interpret natural language commands, bridging the gap between human intent and robotic capabilities.

This demo presents a novel solution that utilizes an LLM to efficiently interface humans and robots by allowing human operators to use freeform prompts without the need of providing detailed instructions following machine readable language. This architecture consists of a server with a customized LLM that interprets commands to derive a high-level plan answering i) Go where? ii) Find what? and iii) Do what?, which are used to generate a structured output consisting of a tuple (destination: $\mathcal{D}$, object: $\mathcal{O}$, action: $\mathcal{A}$). Sharing this output, referred to as DOA hereinafter, with the robot allows it to navigate autonomously to the destination and while efficiently carryout real-time object identification showcasing the efficiency of the proposed human-robot interfacing for real-time control.

II. SYSTEM ARCHITECTURE & HARDWARE

The demo setup consists of a server, a mobile robot, and a demo area. The server computer has a graphics processing unit

This work was supported by the projects EU-CHISTERA MUSE-COM², NSF-AKA CRUISE (GA 24304406), and Infotech-R2D2.

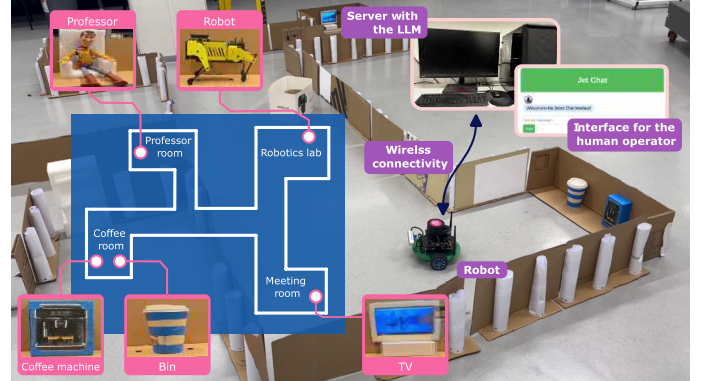


Fig. 1: The robotic platform highlighting the layout and the association of the objects in $\mathcal{O}$ with the destinations in $\mathcal{D}$.

(GPU) of NVIDIA GeForce RTX 2080 Ti uses Ubuntu 20.04 with the robot operating system (ROS) framework. Therein, a Mixtral LLM with 7 billion parameters [2] is deployed to infer the high-level plans from user commands provided through a chat-box type user interface, which are then shared with the robot.

The mobile robot used in this setup is the Waveshare JetBot ROS artificial intelligence (AI) kit, equipped with an NVIDIA Jetson Nano development board [3]. The robot is equipped with an light detection and ranging (LiDAR), an inertial measurement unit (IMU), motor encoders, and an Intel RealSense D415 depth camera for object detection and tracking. For real-time object detection, the robot uses a YOLOv4-tiny model that has been optimized through TensorRT conversion to reduce inference latency [4].

To reflect a realistic environment, we consider a layout consists of a set $\mathcal{O} = \{\text{bin, coffee machine, professor, robot, TV}\}$ of five objects associated with a set $\mathcal{D} = \{\text{coffee room, meeting room, professor's office, robotics lab}\}$ of four destinations as shown in Fig. 1. The choice of the down-scaled confined area is to reduce the unnecessary wondering and thus, limiting the undesirable battery drains at the robot.

III. SYSTEM IMPLEMENTATION

The flow of the demo consists of three steps as discussed next.

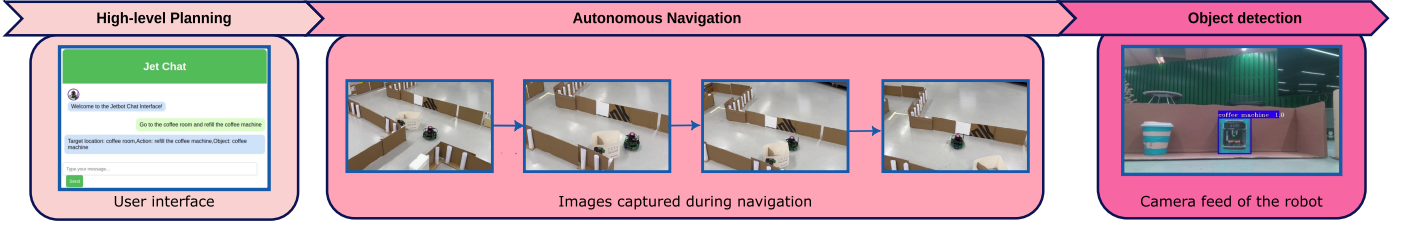


Fig. 2: Flow of the system implementation highlighting the three key steps along the corresponding outputs from an actual demo.

TABLE I: Exemplary prompts and corresponding LLM outputs.

User prompt	Output
Take this file to the professor at the professor's room.	D: professor's room O: professor A: take this file to the professor
Check whether the video feed works in the meeting room.	D: meeting room O: TV A: check whether the video feed works
Switch on the robot.	D: robotics lab O: robot A: switch on
Please make me a coffee.	D: coffee room O: coffee machine A: make coffee

High-level planning: The procedure initiates with a human operator issuing a command as a natural language prompt. The command contains an action (A) related to an object (O) at a location/destination (D) aligning with the context of the demo area, either specified directly or implicitly. LLMs are likely to extrapolate beyond the context of the demo environment and thus, we utilize the *in-context learning* where examples of object-destination associations are fed to the LLM to ensure the responses are valid for the demo. To articulate the response in the desired format of DOA, we adopt *system prompts* [5]. Here, we feed few exemplary prompts and corresponding desired outputs to the LLM so that the LLM can mimic the generation of DOA with the correct format during the inference.

The inference capability of the LLM over the prompts with details to the ones with broader commands is reflected in Table I. Therein, we highlight the D, O, and A with blue, magenta, and teal colors, respectively, when they are explicitly mentioned in the prompt. The first prompt is descriptive while the second and third only specify A with either D or O. The last prompt simply presents A leaving the LLM to interpret both D and O.

Autonomous navigation: Upon receiving the DOA, the robot requires to autonomously navigate to D and detect O. For the navigation, the robot uses a 2D occupancy grid map (OGM) of the demo area having the coordinates of the destinations. First, the D of DOA is mapped to the corresponding coordinates that is published as a `geometry_msgs/PoseStamped` ROS message to the `/move_base_simple/goal` rostopic.

Then, the robot employs the LiDAR sensor to monitor real-time changes in its environment, which are used to update both the OGM and the cost map, a representation of the distances to the obstacle in the environment, generated with TurtleBot3 Navigation package in ROS [6]. With these updates the robot continuously refines its planned trajectory from its current location to D over the shortest possible collision-free path.

Object detection: At D, the robot uses the YOLOv4-tiny model to detect and recognize O in real-time. The YOLOv4-tiny model has been trained on a custom dataset, which was created by positioning the robot at various locations on the platform to capture images of the objects in O from different orientations, followed by manual annotation. The model is then optimized by converting into TensorRT format before the deployment to operate efficiently on the NVIDIA Jetson Nano platform.

The flow of the aforementioned steps along some results captured during a demo are illustrated in Fig. 2.

A. Demo, Resources, and Future Developments

The project's software is available on GitHub at <https://github.com/ICONgroupCWC/LLM-aided-Prompt-to-Control.git>, and a live demo can be viewed at <https://youtu.be/6jZNM8AZ7O0?si=o1vZVbaEHv2ZXkhu>. Future developments will explore towards enhancing LLM inference with planning over multiple tasks as well as supporting multi-robot collaboration enabling coordinated task execution in complex environments.

REFERENCES

- [1] M. Moniruzzaman, A. Rassau, D. Chai, and S. M. S. Islam, "Teleoperation methods and enhancement techniques for mobile robots: A comprehensive survey," *Robotics and Autonomous Systems*, vol. 150, p. 103973, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889021002414>
- [2] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. I. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.
- [3] Waveshare, "Jetbot ros ai kit," Available at <https://www.waveshare.com/jetbot-ros-ai-kit.htm>.
- [4] NVIDIA, "YOLOv4-tiny," Available at https://docs.nvidia.com/tao/tao-toolkit-archive/tao-30-2202/text/object_detection/yolo_v4_tiny.html#deploying-to-deepstream, 2018.
- [5] P. Vedantam, "Prompt engineering for llm applications with langchain," Available at <https://medium.com/@prasanth.vedantam/prompt-engineering-for-llm-applications-with-langchain-6e7aaa5f762a>, Sep. 2023.
- [6] ROBOTIC e-manual, "TurtleBot3 Navigation," Available at <https://emanual.robotis.com/docs/en/platform/turtlebot3/navigation/#navigation>, 2024.